S1_R2_Web Services_PBE I

> Período de desenvolvimento:
>
> **DES-M2H: 07, 14, 21 e 28/08/2025**
> **DES-I1HS:**
> **DES-I1HSB:**

▼ Capacidades

▼ Capacidades Técnicas

Desenvolver API (web services) para integração de dados entre plataformas

▼ Capacidades Socioemocionais

1. Demonstrar autogestão
2. Demonstrar pensamento analítico
3. Demonstrar inteligência emocional
4. Demonstrar autonomia

▼ Conhecimentos

2. Web Services
   2.1. Definição
   2.2. REST
   2.2.1. Recursos
   2.2.2. Semântica da URL REST
   2.3. Padrão JSON
   2.3.1. Sintaxe básica
   2.3.2. Tipos de dados
   2.3.3. Formatação
   2.3.4. Coleção de objetos JSON
   2.4. XML
   2.4.1. Sintaxe básica
   2.4.2. Tipos de dados
   2.4.3. Formatação

▼ O que é um Web Service?

> Um **Web Service** é um conjunto de métodos ou funcionalidades que podem ser utilizados e invocados por outros programas utilizando tecnologias Web.

É utilizado para transferir dados através de protocolos de comunicação para diferentes plataformas, independentemente das linguagens de programação utilizadas nessas plataformas.

▼ Funcionamento

Os Web services funcionam independente de:

- Sistema operativo,
- Plataforma de hardware ou
- Linguagem de programação de suporte Web.

Estes transmitem apenas informação, ou seja, não são aplicações Web que suportam páginas que podem ser acessadas através de navegadores Web.

Os Web services permitem reutilizar sistemas já existentes numa organização e acrescentar-lhes novas funcionalidades sem que seja necessário criar um sistema a partir do zero. Assim, é possível melhorar os sistemas já existentes, integrando mais informação e novas funcionalidades de forma simples e rápida.

▼ Como aplicações com linguagens diferentes se comunicam?

É necessário uma linguagem intermediária que garanta a comunicação entre a linguagem do *Web Service* e o sistema que faz o pedido ao *Web Service*. Para tal, existem protocolos de comunicação como:

- **SOAP Simple Object Access Protocol** (Protocolo Simples de Acesso a Objetos) e o
- **REST Representational State Transfer** (Transferência Representacional de Estado).

O protocolo SOAP utiliza XML para enviar mensagens e, geralmente, serve-se do protocolo HTTP para transportar os dados. Associado ao protocolo SOAP está o documento WSDL (Web Service Definition Language) que descreve a localização do *Web Service* e as operações que dispõe. Além disso, fornece a informação necessária para que a comunicação entre sistemas seja possível.

O REST é um protocolo de comunicação mais recente que surgiu com o objetivo de simplificar o acesso aos Web services. Este baseia-se no protocolo HTTP e permite utilizar vários formatos para representação de dados, como JSON (um dos mais utilizados), XML, entre outros.

Assim, uma das grandes vantagens do REST é a sua flexibilidade, já que não limita os formatos de representação de dados. O protocolo REST é também utilizado quando a performance é importante, uma vez que é um protocolo ágil e com a capacidade de transmitir dados diretamente via protocolo HTTP.

▼ Benefícios dos Web Services

- **Integração simplificada**: Utilizam XML/JSON e HTTP, permitindo comunicação fácil entre sistemas de diferentes plataformas (Windows, Linux) e linguagens (Java, Python), sem necessidade de detalhes sobre o funcionamento interno.
- **Reutilização de código**: Um Web service é desenvolvido uma vez e pode ser usado repetidamente por várias aplicações e plataformas, com diferentes objetivos.
- **Desenvolvimento mais rápido**: Reduz o tempo de construção de sistemas, pois não exige desenvolvimento do zero e facilita a adição de funcionalidades.
- **Maior segurança**: Evita acesso direto ao banco de dados, protegendo os dados do sistema.
- **Redução de custos**: Aproveita protocolos e infraestrutura web existentes, eliminando a necessidade de soluções personalizadas caras para integração.

▼ O que é uma API?

Uma interface de programação de aplicação (API) define as regras que você precisa seguir para se comunicar com outros sistemas de software. Os desenvolvedores expõem ou criam APIs para que outras aplicações possam se comunicar com suas aplicações programaticamente. Por exemplo, a aplicação de planilha de horas expõe uma API que solicita o nome completo de um funcionário e um intervalo de datas. Ao receber essas informações, processa internamente a planilha de horas do funcionário e retorna o número de horas trabalhadas nesse intervalo de datas.

Clientes

Clientes são usuários que desejam acessar informações da Web. O cliente pode ser uma pessoa ou um sistema de software que usa a API. Por exemplo, desenvolvedores podem escrever programas que acessam dados meteorológicos de um sistema meteorológico. Ou você pode acessar os mesmos dados do seu navegador ao visitar o site meteorológico diretamente.

Recursos

Recursos são as informações que diferentes aplicações fornecem aos seus clientes. Os recursos podem ser imagens, vídeos, textos, números ou qualquer tipo de dado. A máquina que fornece o recurso ao cliente também é chamada de servidor. As organizações usam APIs para compartilhar recursos e fornecer

serviços da Web, mantendo a segurança, o controle e a autenticação. Além disso, as APIs ajudam a determinar quais clientes têm acesso a recursos internos específicos.

Fonte:

O que é a API RESTful? – Explicação sobre a API RESTful – AWS

Saiba o que é a API RESTful, como e por que as empresas usam APIs RESTful e como usar o API Gateway com a AWS.

https://aws.amazon.com/pt/what-is/restful-api/

▼ REST

REST é um estilo de arquitetura para desenvolvimento de Web Services que determina condições sobre como deve funcionar, mais leve e simples do que o modelo baseado em SOAP.

▼ Benefícios do REST

- **Escalabilidade**: APIs REST são fáceis de expandir porque otimizam a comunicação entre cliente e servidor. Como não guardam informações de requisições anteriores, os servidores ficam menos sobrecarregados. Além disso, o uso de cache reduz a necessidade de novas interações, evitando problemas de desempenho.
- **Flexibilidade**: APIs REST permitem que cliente e servidor funcionem separadamente. Isso simplifica os componentes do servidor, que podem ser atualizados sem impactar o cliente. Alterações no banco de dados ou na tecnologia do servidor, por exemplo, não exigem mudanças na aplicação do cliente.
- **Independência**: APIs REST não dependem de uma tecnologia específica. Cliente e servidor podem ser desenvolvidos em diferentes linguagens de programação, e mudanças na tecnologia de um lado não afetam a comunicação entre eles.

▼ Princípios do REST

- Utiliza os **métodos HTTP**: GET, POST, PUT, DELETE
- Utiliza **URLs como identificadores de recursos**
- É **stateless**: o servidor não armazena o estado do cliente
- Respostas em **JSON** ou **XML**, comumente JSON

Métodos HTTP comuns:

Método	Ação
GET	Buscar um recurso

Método	Ação
POST	Criar um novo recurso
PUT	Atualizar um recurso existente
DELETE	Excluir um recurso

▼ Recursos REST

No REST, recursos são os elementos que a aplicação expõe e manipula via Web Service. Podem representar qualquer entidade da aplicação: usuários, produtos, pedidos, etc.

Exemplos de recursos:

- `/usuários` → lista de usuários
- `/produtos` → lista de produtos
- `/pedidos` → lista de pedidos

Cada recurso é identificado por uma URL única

▼ Semântica da URL REST

A **semântica da URL** diz respeito à maneira como a URL é construída para refletir a ação ou o recurso sendo acessado.

Boas práticas REST:

- Procure usar os **substantivos** das URLs no **plural**
- Não usar verbos na URL (os verbos estão nos métodos HTTP)

Exemplos práticos:

Ação	Verbo HTTP	URL RESTful	Significado
Buscar todos os usuários	GET	`/usuarios`	Retorna todos os usuários
Buscar um usuário específico	GET	`/usuarios/5`	Retorna o usuário com ID 5
Criar um novo usuário	POST	`/usuarios`	Cria um novo usuário
Atualizar usuário existente	PUT	`/usuarios/5`	Atualiza os dados do usuário 5
Excluir um usuário	DELETE	`/usuarios/5`	Remove o usuário 5

💡

> Manual do proprietário de APIs: https://www.redhat.com/rhdc/managed-files/is-api-owners-manual-ebook-556485-202311-ptbr.pdf

attachment:7ceb5a95-7fdb-47ba-9290-ed5ac400edc1:is-api-owners-manual-ebook-556485-202311-ptbr.pdf

---

▼ Padrão JSON

> **JSON** (JavaScript Object Notation - Notação de Objetos JavaScript) é uma formatação leve de troca de dados. Para seres humanos, é fácil de ler e escrever.

É um formato de arquivo de padrão aberto para armazenar dados de maneira organizada e legível, ao mesmo tempo que facilita o acesso. Os dados são apresentados em um formato de texto básico, que consiste em pares "**chave/valor**".

JSON está constituído em duas estruturas:

- Uma coleção de pares **nome**/**valor**. Em várias linguagens, isto é caracterizado como um *object*, record, struct, dicionário, hash table, keyed list, ou arrays associativas.
- Uma lista ordenada de valores. Na maioria das linguagens, isto é caracterizado como uma *array*, vetor, lista ou sequência.

Muitos sites, Web Services e APIs usam JSON para compartilhamento de dados.

Portanto, amplamente utilizado para comunicação assíncrona entre navegador e servidor, em particular como um substituto para XML.

▼ Como surgiu o JSON?

JSON (JavaScript Object Notation) é um formato de dados que é usado para armazenar e trocar informações entre sistemas. Ele foi criado no início dos anos 2000 por **Douglas Crockford**, um programador e consultor de tecnologia.

📄História e Linha do Tempo do JSON

Anos 1990 – contexto

- O **JavaScript** foi criado por Brendan Eich em 1995 na Netscape.
- Na época, para comunicação entre cliente e servidor, o **XML** era muito usado, mas era **pesado e complexo**.
- Os desenvolvedores começaram a usar objetos JavaScript como uma forma leve de representar dados.

2001 – Surgimento do JSON (oficial)

- **Douglas Crockford**, um programador e autor, **padronizou o uso de objetos JavaScript como formato de dados** e batizou de **JSON**.
- Ele registrou o domínio **json.org** para divulgar a ideia.
- Publicou uma **especificação informal** explicando como JSON funcionava.

2002 – Primeiras implementações

- Começou a surgir **bibliotecas em várias linguagens** para trabalhar com JSON.
- Douglas Crockford escreveu um **parser JSON para JavaScript e outras linguagens**.

2005 – JSON se populariza com AJAX

- O termo **AJAX (Asynchronous JavaScript and XML)** ficou famoso após um artigo de Jesse James Garrett.
- Mas muitos desenvolvedores começaram a usar **JSON no lugar do XML** em requisições AJAX, por ser mais simples.

2006 – JSON é adotado por frameworks e APIs

- Frameworks como jQuery facilitam o uso de JSON.
- A comunicação via JSON se tornou padrão em aplicações web modernas.
- JSON começa a ser amplamente usado em **APIs RESTful**.

2013 – JSON vira padrão oficial

- O **ECMA International** publica o padrão **ECMA-404**, definindo formalmente o JSON como um padrão internacional.
- JSON também passa a ser documentado na **RFC 7159** pelo IETF (Internet Engineering Task Force).

2017 – RFC 8259 (atual)

- A **RFC 8259** atualiza e consolida a especificação do JSON como padrão IETF.
- Define regras formais sobre strings, números, objetos e arrays no JSON.

▼ Sintaxe

A sintaxe do JSON é baseada em dois elementos fundamentais:

- **Objetos**: representados por `{}` (chaves);
- **Arrays**: representados por `[]` (colchetes);

Cada objeto é formado por **pares chave/valor**. A **chave** deve ser uma **string** (entre aspas duplas) e o **valor** pode ser de diversos.

▼ Exemplo básico JSON

```
{
    "nome": "Izaias",
    "idade": 30,
    "ativo": true,
    "cursos": ["JavaScript", "Node.js", "Java"]
}
```

▼ Exemplo com array (lista)

```
{
    "cursos": ["HTML", "CSS", "JavaScript"]
}
```

▼ Formatação

JSON deve seguir uma formatação **estrita**. Alguns erros comuns:

- Usar aspas simples ('') em vez de duplas.
- Usar vírgula depois do último item de um objeto ou array.

```
{
    "nome": "Lucas",
    "idade": 31,
    "profissoes": ["Desenvolvedor", "Professor"]
}
```

▼ Coleção de objetos JSON

Uma **coleção de objetos JSON** normalmente é representada como **um array contendo vários objetos**.

```
[
    {
        "nome": "Pedro",
        "idade": 18
    },
    {
        "nome": "Camila",
        "idade": 32
    },
    {
        "nome": "Luana",
        "idade": 16
    }
]
```

💡

Curiosidades

- JSON é um subconjunto da notação de objetos do JavaScript, mas **não depende de JavaScript** para ser usado.
- Apesar do nome, **JSON não é exclusivo da linguagem JavaScript**. Ele é usado em linguagens como **Python, Java, PHP, Ruby, Go, C#, etc.**

JSON
JSON (JavaScript Object Notation - Notação de Objetos JavaScript) é uma formatação leve de troca de dados. Para seres humanos, é fácil de ler e escrever. Para máquinas, é fácil de interpretar e gerar. Está baseado em um
https://www.json.org/json-pt.html

▼ Padrão XML

O XML, **sigla para Extensible Markup Language (Linguagem de Marcação Extensível)**, é uma linguagem de marcação que define um conjunto de regras para codificar documentos em um formato legível tanto para humanos quanto para máquinas.

Foi desenvolvido pela **W3C (World Wide Web Consortium)** em **1996**, com a primeira recomendação oficial publicada em **1998**. Ele surgiu da necessidade de ter um

formato mais flexível e extensível para representar dados de forma estruturada, principalmente para **troca de informações entre sistemas diferentes na internet**.

Inspirado no **SGML (Standard Generalized Markup Language)**, o XML foi criado para ser **mais simples, legível e utilizável em diferentes plataformas**, tornando-se um padrão importante para integração de sistemas, especialmente antes da popularização de formatos como JSON.

O XML é amplamente utilizado em:

- **Troca de dados** entre aplicações;
- **Configuração de softwares** (ex.: `web.config`, `pom.xml`);
- **Armazenamento de dados estruturados**;
- **Documentação e comunicação** entre diferentes tecnologias .

O XML possui regras claras de estruturação dos dados. Sua sintaxe é baseada em ***tags***, muito parecida com HTML, porém com finalidade **semântica e não visual**.

Exemplo:

```
<livros>
  <livro>
      <titulo>Aprendendo XML</titulo>
      <autor>Pedro Silva</autor>
      <ano>2025</ano>
      <preco moeda="BRL">49.90</preco>
  </livro>
  <livro>
      <titulo>Aprendendo JAVA</titulo>
      <autor>Pedro Silva</autor>
      <ano>2018</ano>
      <preco moeda="BRL">109.90</preco>
  </livro>
</livros>
```

▼ Regras básicas da sintaxe XML:

1. **Um único elemento raiz** (no exemplo: `<livro>`);
2. **As tags devem ser bem formadas** (abertura e fechamento corretos);
3. **Sensível a maiúsculas e minúsculas** (`<Livro>` ≠ `<livro>`);
4. **Atributos devem estar entre aspas** (`moeda="BRL"`);
5. **Comentários** são feitos com `<!-- comentário -->`.

▼ Tipos de dados

O XML **não possui tipos de dados nativos** como JSON (número, booleano etc.). Por padrão, **tudo é texto (string)**.

No entanto, é possível **impor ou validar tipos de dados** usando esquemas como:

1. DTD (Document Type Definition)

Define regras estruturais, mas com limitações em tipos.

2. XML Schema (XSD)

Permite definir tipos de dados como:

- `xs:string`
- `xs:integer`
- `xs:boolean`
- `xs:date`, entre outros.

Exemplo com XSD:

```
<?xml version="1.0" encoding="UTF-8"?>
<produto xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
                xsi:noNamespaceSchemaLocation="produto.xsd">
      <codigo>101</codigo>
      <nome>Teclado Mecânico</nome>
      <preco>199.90</preco>
      <estoque>25</estoque>
      <disponivel>true</disponivel>
</produto>
```

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
   <xs:element name="produto">
   <xs:complexType>
      <xs:sequence>
         <xs:element name="codigo" type="xs:integer"/>
         <xs:element name="nome" type="xs:string"/>
         <xs:element name="preco" type="xs:decimal"/>
         <xs:element name="estoque" type="xs:integer"/>
         <xs:element name="disponivel" type="xs:boolean"/>
      </xs:sequence>
   </xs:complexType>
```

```
  </xs:element>
</xs:schema>
```

▼ Formatação

A formatação no XML, refere-se a forma como o conteúdo do arquivo é escrito, organizado e apresentado, visando clareza e legibilidade.

Apesar dos sistemas e dispositivos poderem interpretar o XML sem formação, ela é essencial para que facilite a leitura por seres humanos.

A formatação XML é baseada em uma **estrutura hierárquica** e **indentação clara**, o que facilita a leitura e a interpretação tanto por humanos quanto por máquinas.

Boas práticas de formatação:

Embora opcional, é boa prática iniciar o XML com:

```
<?xml version="1.0" encoding="UTF-8"?>
```

- Indicar a versão do documento.
- Deve estar sempre na primeira linha.

▼ Usar **indentação com espaços ou tabulações** para indicar a hierarquia;

- A indentação mostra a hierarquia entre os elementos.

```
<livro>
    <titulo>XML em Detalhes</titulo>
    <autor>Joana Prado</autor>
</livro>
```

▼ Tags com nomes descritivos

- Exemplo de como nomear as tags: `<nomeCompleto>` ao invés de `<n>` ou `<x>`);
- Utilizar nomes que façam sentido com o item referenciado.
- Evitar espaços e acentos em nomes de elementos;

```
<cpf>12345678900</cpf> <!-- Bom -->
<n1>12345678900</n1>   <!-- Ruim: pouco informativo -->
```

▼ Padronizar o uso de **aspas duplas para atributos**;

```
<telefone tipo="celular">(11) 91234-5678</telefone>
```

▼ **Fechamento das tags:**

```
<tag></tag> <!-- Normal -->
<tag />     <!-- Autofechamento (vazio) -->
```

💡

O que é XML? Guia completo da linguagem universal de dados

O XML é uma linguagem de marcação muito famosa e útil dentro da tecnologia e até fora desse meio, venha descobrir tudo sobre ela aqui.

https://blog.tecnospeed.com.br/historico-do-xml-e-seus-fragmentos/